CSCI4730/6730 – Operating Systems

Project #3

Due date: 11:59pm, 11/17/2024

Description

In this project, you will be implementing a virtual memory simulator in order to understand the behavior of a page table and the page replacement algorithms. Virtual memory allows the execution of processes that are not completely in memory. This is achieved by page table and page replacement techniques. A page table stores the mapping information between virtual addresses and physical addresses. A page fault mechanism by which the memory management unit (MMU) can ask the operating system (OS) to bring in a page from the swap disk. The system we are simulating is **16-bit machine (8 bits for index and 8 bits for offset).**

Part 1: Page Table – (50%)

The page table structure that we will implement is a linear page table. Each page table entry contains a physical frame number (PFN), a valid bit, and a dirty bit. The size of main memory (# of physical memory frames) is configurable via command-line arguments. The size of virtual memory (# of virtual pages), and the number of process are specified in the input file. The machine we are simulating supports up to 256 pages, 256 frames, and 8 processes.

Initially, all physical frames are free. Our page allocation policy simply hand out the free physical frame until the free-frame list becomes empty. When the free-frame list is empty, you should find a victim page using page replacement policy.

We also simulate the placement of virtual pages on the swap disk. We keep track of the number of swap disk read and write operations that are needed to handle the page faults. We maintain “dirty” bit on the page table and swap out the dirty page when it is replaced.

In this project, you will need to handle two types of requests. The request format is the following:

***[pid] [R** or **W] [Virtual Address] [Byte** (if W)]*

1. **Read req**: It reads 1 byte of the memory.
2. **Write req**: It writes 1 byte to the memory. For simplicity, we use a single character to fill-out the entire page. For example, if the request is "1 W 0x03D2 H", it will fill out the entire page from 0x0300 to 0x03FF (256 bytes) with "H". We assume that the entire process memory is initialized with "A".

In this project, you **do not** need to directly access data structure for "page table", "inverted page table", or "swap disk". Instead, you can simply use APIs to access them. All APIs are defined in "API.h" header file.

1. To access the page table (convert virtual page number (VPN) to physical frame number (PFN))
   A. **PTE read_PTE (int pid, int VPN);**
   B. **void write_PTE(int pid, int VPN, PTE pte);**

2. To access the inverted page table (convert physical frame number (PFN) to virtual page number (VPN) and process id (pid)):
   A. **IPTE read_IPTE (int PFN);**
   B. **void write_IPTE (int PFN, IPTE ipte);**

3. To access the swap disk:
   A. **void swap_in(int pid, int VPN, int PFN);** // read swap disk: swap space -> physical memory
   B. **void swap_out(int pid, int VPN, int PFN);** // write swap disk: physical memory → swap space

The following are details of the data structure:

- Data structure for the page table is defined in vm.h
  - PTE is a page table entry. It contains virtual to physical mapping information for each virtual pages including [physical frame number], [valid bit], and [dirty bit].

```
typedef struct {
  int PFN;
  bool valid;
  bool dirty;
} PTE;
```

  - Each process has its own page table.

- Invert page table is necessary to identify process id and virtual page number for each physical frame. You need this information during swapping procedure.

```
typedef struct {
  int pid;
  int VPN;
} IPTE;
```

Implementation guide line:

1. You will need to implement the following three functions in "pagetable.c" file.
   A. **int MMU(int pid, int virtAddr, char reqType, bool *hit)**
      i. This function convert virtual memory address into physical memory address. It also needs to check the provided virtual address is valid or not.
   B. **int get_PFN(int pid, int VPN, char reqType)**
      i. This function check the page table. If the requested page is available and valid, return PFN. If the page is not valid, return -1.
   C. **int pagefault_handler(int pid, int VPN, char reqType)**
      i. This function handles the page fault. If a free frame is available, simply use it to load a request page to physical memory. Note that, "get_freeframe()" function to get free frame is provided and you don't need to implement it. If no free frame is available, call "find_replacement" function to find a victim frame from the physical memory.
      ii. To read a page from swap disk, use "swap_in" function. To write a page from the memory to swap disk (when it is dirty), use "swap_out" function.

2. Currently, only ZERO replacement algorithm is ready to use that always replaces the frame 0 for the page fault. You will be implementing FIFO, LRU, and CLOCK in the next part.

Part 2: Page Replacement Algorithm– (50%, FIFO:10%, LRU:20%, CLOCK:20%, CLOCK-PRO:20% of extra credit)

The current simulator only supports ZERO page replacement algorithm that always replace the frame 0.
In this part, you will implement three page replacement algorithms, First-in-first-out (FIFO) and Least-recently-used (LRU) algorithms, and second chance algorithm (CLOCK).

Additionally, if you choose to implement the CLOCK-PRO algorithm, you can earn up to 20% extra credit. Details on CLOCK-PRO, including data structures and implementation, are available in the following paper:
https://www.usenix.org/legacy/publications/library/proceedings/usenix05/tech/general/full_papers/jiang/jiang.pdf

- All replacement functions are declared in pagetable.c file, but they are empty now.
- The simulator requires a command-line argument to specify a replacement policy to use.
    - 0: ZERO
    - 1 : FIFO
    - 2 : LRU
    - 3 : CLOCK
    - 4 : CLOCK-PRO
- You can use a "doubly linked list" to maintain the LRU list. The files "list.c" and "list.h" provide a doubly linked list implementation. However, feel free to use your own or preferred linked list implementation if you wish.
    - Define head node with “struct Node”. This should be initialized as NULL.
    - “list_insert_head()” to insert a new item into the head of the list
    - “list_insert_tail()” to insert an item into the tail of the list.
    - “list_remove()” to remove an item.
    - “list_print()” to print the list.
    - list_test.c is a test implementation for the list

Command-line arguments

Our virtual memory simulator requires three command line arguments.

1. Number of frames in the physical memory (between 1 and 256)
2. Page replacement policy (0 - ZERO, 1 - FIFO, 2 - LRU, 3 – CLOCK, 4 – CLOCK_PRO)
3. Input file that contains
    A. The number of processes and virtual pages
    B. reference string
4. ex) ./vm 4 0 ./example_inputs/belady_input.txt

Input Generator

The input format for your simulator will be in the following format:

*[**pid**] [**R** or **W**] [**Virtual Address**] [**Byte** (if W)]*

You can find example inputs, including belady sequence and reference string we used in the class in "example_input" folder.
The first line of the input file indicates "# of pages" and "# of processes"
You may use a provided input generation tool "input_gen" to generate random inputs.

- $./input_gen [# of page] [# of process] [# of requests]

Example of output (belady input with 6 virtual pages, 3 physical frames, FIFO):

```
$ ./vm 3 1 ./example_inputs/belady_input.txt
# PAGES: 6, # FRAMES: 3, # PROCS: 1, Replacement Policy: 1 - FIFO
[pid 0, W] 0x0100 --> 0x0000: a [miss]
[pid 0, W] 0x0200 --> 0x0100: b [miss]
[pid 0, W] 0x0300 --> 0x0200: c [miss]
[pid 0, W] 0x0400 --> 0x0000: d [miss]
[pid 0, R] 0x0100 --> 0x0100: a [miss]
[pid 0, R] 0x0200 --> 0x0200: b [miss]
[pid 0, W] 0x0500 --> 0x0000: e [miss]
[pid 0, R] 0x0100 --> 0x0100: a [hit]
[pid 0, R] 0x0200 --> 0x0200: b [hit]
[pid 0, R] 0x0300 --> 0x0100: c [miss]
[pid 0, R] 0x0400 --> 0x0200: d [miss]
[pid 0, R] 0x0500 --> 0x0000: e [hit]
====================================
Request: 12
Page Hit: 3 (25.00%)
Page Miss: 9 (75.00%)
Swap Read: 9
Swap Write: 4
```

Important! You should remove all debugging messages before the submission. TA will use "diff" to compare your output with the solution output. You could have a penalty if your code emits anything other than the defined output.

Tip! You can use "debug(..)" instead of "printf(..)" to print out your debugging messages. It will print out the messages in the screen if you comment out "#define NDEBUG" in vm.h file. You can simply enable "#define NDEBUG" before the submission to remove the debugging messages.

Testing

You may use "vm_reference" executable to generate reference output and compare it with yours. Note that it is not yet fully tested and could have bugs and can crash.
$./vm_reference 3 1 ./example_inputs/belady_input.txt

Extra Credit #1: CLOCK-PRO replacement algorithm (up to 20%)

See "Part 2: Page Replacement Algorithm".

Extra Credit #2: Bug Bounty – (up to 10%)

The reference output program has not been fully tested and may contain some bugs. If you identify a bug in the reference code, please report it. The first person to report each unique bug will receive 5% extra credit (up to a maximum of 10%). Only the first person to report a given bug will receive credit.

To report a bug, email kyuhlee@uga.edu with the subject line "CSCI6730 – Project 3 Bug Report." Your report should include a description of the bug, minimal inputs that can reproduce the bug, the root cause, and a suggested fix.

I will review reports and post any confirmed bugs on ELC.

Please note that minor typos in output strings or simple bugs that do not impact correctness will not be counted as bugs.

Submission

Submit a tarball file using the following command:
% tar czvf p3.tar.gz report.pdf Makefile pagetable.c (if you have any other files added or modified, include them)

1. Report file with:
    a. Your name
    b. List what you have done and how you tested them. So that you can be sure to receive credit for the parts you've done.
    c. Explain your design of data structures.
    d. Report should be **PDF format. Only PDF will get points.**
2. **Your code should be compiled in odin machine. Any compilation error on Odin will lead to 0 point.**
3. Submit a tarball through ELC.